

Parte 1: Investigación Teórica

Byron León Ruiz

1. ¿Qué función cumple HTML en la creación de menús?

En la creación de menús, HTML cumple la función estructural fundamental. Vos usás HTML para definir qué opciones de navegación existen y cómo se agrupan. Sin HTML, no hay menú, porque es el lenguaje que le dice al navegador: “acá hay un botón”, “acá hay una lista de enlaces”, “este es el contenedor del menú”. Los elementos típicos que usamos son `<nav>` para envolver todo el menú, `<ul>` y `<li>` para listar las opciones, `<a>` para los enlaces, y `<button>` o `<label>` para el disparador del menú hamburguesa. En definitiva, HTML aporta la arquitectura base, el esqueleto sobre el cual después aplicás estilos y comportamiento.

2. ¿Qué se puede hacer únicamente con CSS en un menú?

Con CSS solo podés lograr muchas cosas visuales y algunas interacciones básicas. Por ejemplo, podés ocultar y mostrar un menú usando el `:hover` o el truco del `:checked` con un checkbox, tal como viste en la teoría del menú hamburguesa que usaba un `<input type="checkbox">` y el selector `:checked`. También podés cambiar colores, tamaños, animar transiciones (como que el menú se deslice suavemente), hacer que el menú se adapte a diferentes tamaños de pantalla con `@media`, y aplicar efectos visuales como sombras, bordes o fondos. Lo que no podés hacer con CSS solo es manejar interacciones complejas como detectar un clic fuera del menú para cerrarlo, o agregar y sacar elementos del menú de forma dinámica después de que la página ya cargó.

3. ¿Qué aporta JavaScript que no se puede hacer con HTML y CSS?

JavaScript aporta la capacidad de manejar eventos complejos y modificar el DOM en tiempo real. En un menú, lo que JS te permite y que no podés lograr con HTML+CSS incluye: detectar un clic fuera del menú para cerrarlo automáticamente, manejar múltiples niveles de submenús que se abren y cierran sin depender de un checkbox por cada uno, agregar opciones al menú dinámicamente después de que el usuario hizo alguna acción, guardar el estado del menú (abierto o cerrado) en el almacenamiento local para que persista al recargar la página, y controlar el foco del teclado para hacer el menú accesible para personas que usan solo el teclado. Básicamente, JS te da control total sobre el comportamiento en respuesta a cualquier evento, no solo clics y hover.

Parte 2: Análisis de Limitaciones

HTML – Limitaciones: HTML es puramente estático. Una vez que la página cargó, los elementos HTML no cambian por sí solos. No podés hacer que un menú se

oculte o se muestre solo con HTML, ni podés crear interacciones. Tampoco podés estructurar un menú que se adapte a diferentes roles de usuario sin recargar la página desde el servidor. Interacción: Con HTML solo, la única “interacción” que tenés son los enlaces que llevan a otra página o los botones dentro de un formulario. No hay forma de que un clic en un botón muestre u oculte algo sin ayuda de CSS o JS. Contenido dinámico: HTML no puede generar ni modificar contenido después de que la página se renderizó. Si necesitás que las opciones del menú cambien según la hora del día o según lo que el usuario eligió antes, con HTML solo no podés.

CSS – Tipo de interacciones: CSS permite interacciones básicas usando pseudo-clases como `:hover` (cuando el mouse pasa por encima), `:focus` (cuando un elemento recibe foco) y `:checked` (cuando un checkbox o radio está activado). Con esas podés mostrar/ocultar elementos, cambiar colores, aplicar transformaciones. Limitaciones frente a eventos: CSS no puede detectar eventos como “clic fuera de un elemento”, “presionar la tecla Escape”, “doble clic”, o “mantener presionado”. Tampoco puede ejecutar lógica condicional compleja (si pasa esto Y esto otro, entonces hacé aquello). Además, la interacción con `:checked` requiere que el checkbox esté en una posición específica en el HTML (antes del elemento que querés controlar) y no funciona si el checkbox y el menú están en ramas del DOM muy separadas. Problemas en menús complejos: Si tenés un menú con submenús anidados de varios niveles, con CSS solo se vuelve muy difícil de manejar. Por ejemplo, para que un submenú se cierre cuando el mouse sale de toda el área, necesitás estructuras HTML muy específicas y el resultado suele ser poco confiable, especialmente en pantallas táctiles donde `:hover` no funciona como esperás.

JavaScript – Problemas que soluciona: JS soluciona la falta de interacción avanzada de HTML y CSS. Permite cerrar el menú al hacer clic fuera, manejar la apertura y cierre de múltiples submenús sin conflictos, agregar animaciones controladas por JavaScript (no solo transiciones CSS), recordar el estado del menú, y hacer que el menú sea accesible para lectores de pantalla y navegación por teclado. Desventajas: JS agrega complejidad al código; tenés que escribir más líneas y eso aumenta las chances de errores. Depende de que el usuario tenga JS habilitado en su navegador (aunque hoy es raro que alguien lo desactive). También puede afectar el rendimiento si está mal optimizado (por ejemplo, si escuchás eventos de redimensionado de ventana sin control). Además, si el JS no carga por un problema de red o un error en el código, el menú puede quedar totalmente inoperable. ¿Es siempre necesario? No, no siempre es necesario. Si tu menú es muy simple y solo necesitás que se oculte/muestre al hacer clic en un botón y no te preocupa la funcionalidad de “clic fuera”, podés usar perfectamente el truco del checkbox con CSS. También si tu menú es un menú horizontal que se adapta a celular usando `@media` y se transforma en otra cosa sin necesidad de ocultarlo, podés prescindir de JS. La regla de oro es: usá JS solo cuando la interacción que necesitás no se pueda resolver limpia y confiablemente solo con CSS.

Parte 3: Cuadro Comparativo

Aspecto	HTML	CSS	JavaScript
Función principal	Define la estructura y el contenido del menú (elementos, textos, enlaces).	Define la presentación visual (colores, tamaños, posición, animaciones).	Define el comportamiento dinámico (abrir/cerrar, responder a eventos complejos).
Tipo de interacciones que permite	Ninguna. Solo enlaces a otras páginas.	Básicas: :hover, :focus, :checked.	Avanzadas: clic fuera, tecla Escape, eventos táctiles personalizados.
¿Puede ocultar/mostrar elementos?	No.	Sí, mediante display: none, visibility, opacity combinado con pseudo-clases.	Sí, modificando directamente el estilo o agregando/sacando clases CSS.
¿Necesita evento del usuario?	No aplica.	Sí, requiere pseudo-clases que responden a eventos del usuario.	Sí, requiere addEventListener para escuchar eventos.
Dependencia de la estructura del HTML	Es la base misma de la estructura.	Alta. Los selectores como ~ o + dependen de la posición exacta de los elementos.	Baja. Podés usar querySelector para encontrar elementos sin importar tanto su posición relativa.
Capacidad para manejar contenido dinámico	Ninguna.	Ninguna.	Sí, podés agregar o quitar opciones del menú después de que la página cargó.

Accesibilidad para lectores de pantalla	Buena si usás las etiquetas correctas (<nav>, , <a>).	Puede ayudar (colores de contraste) o perjudicar (display: none oculta también para lectores).	Buena si manejas correctamente aria-expanded y el foco del teclado.
Rendimiento	Mínimo.	Mínimo, salvo animaciones muy pesadas.	Puede ser un problema si está mal escrito (muchos eventos, reflows innecesarios).

Parte 4: Análisis de un Menú Visto en Clase

HTML: La estructura usa un `<input type="checkbox" id="menu-toggle">` escondido, un `<label for="menu-toggle">` que actúa como botón con el ícono ☰, un `<header>` que contiene al label, y un `<nav>` que contiene los ítems del menú (cuatro `<div>` con las letras A, B, C, D). La ubicación de los elementos es clave: el checkbox y el label están antes que el `<nav>` en el código, porque el selector `~` en CSS necesita que el elemento a controlar esté después del checkbox al mismo nivel.

CSS: El CSS primero oculta el checkbox con `display: none`. Luego, el botón `.menu-btn` tiene `display: none` en pantallas grandes para que no se vea. Dentro de un `@media (max-width: 480px)`, el botón cambia a `display: block` (se muestra) y el `<nav>` tiene `display: none` (se oculta). Por último, la regla `#menu-toggle:checked ~ nav` hace que cuando el checkbox está activado, el `<nav>` que viene después se muestre (en este caso, con `display: grid`). También se agregaron estilos para los ítems del menú: fondo oscuro, texto blanco, padding, borde inferior y efecto hover que cambia el fondo a gris más claro.

Parte 6: Reflexión Final

Para mí, la tecnología que me resultó más útil en este desarrollo es CSS. Si bien JavaScript es muy potente y sin él no podría hacer el cierre con clic fuera ni la tecla Escape, la realidad es que el 80% del menú hamburguesa funciona perfectamente solo con HTML y CSS. El truco del checkbox con `:checked` te permite crear componentes interactivos sin depender de ningún script, lo que hace que la página sea más liviana, más rápida y más robusta ante fallos de red o errores de código. Además, CSS te da las herramientas para que el menú se vea bien en cualquier

dispositivo con @media, y para que las animaciones sean suaves y no traben el renderizado. Yo considero que, siempre que puedas resolver un problema sin JS, es mejor hacerlo. JS hay que usarlo solo para lo que realmente no se puede hacer de otra manera, como en este caso el clic fuera o la gestión del foco avanzada. Aprendí que un buen desarrollador no es el que mete JS en todos lados, sino el que sabe elegir la herramienta justa para cada necesidad.